Lab 07

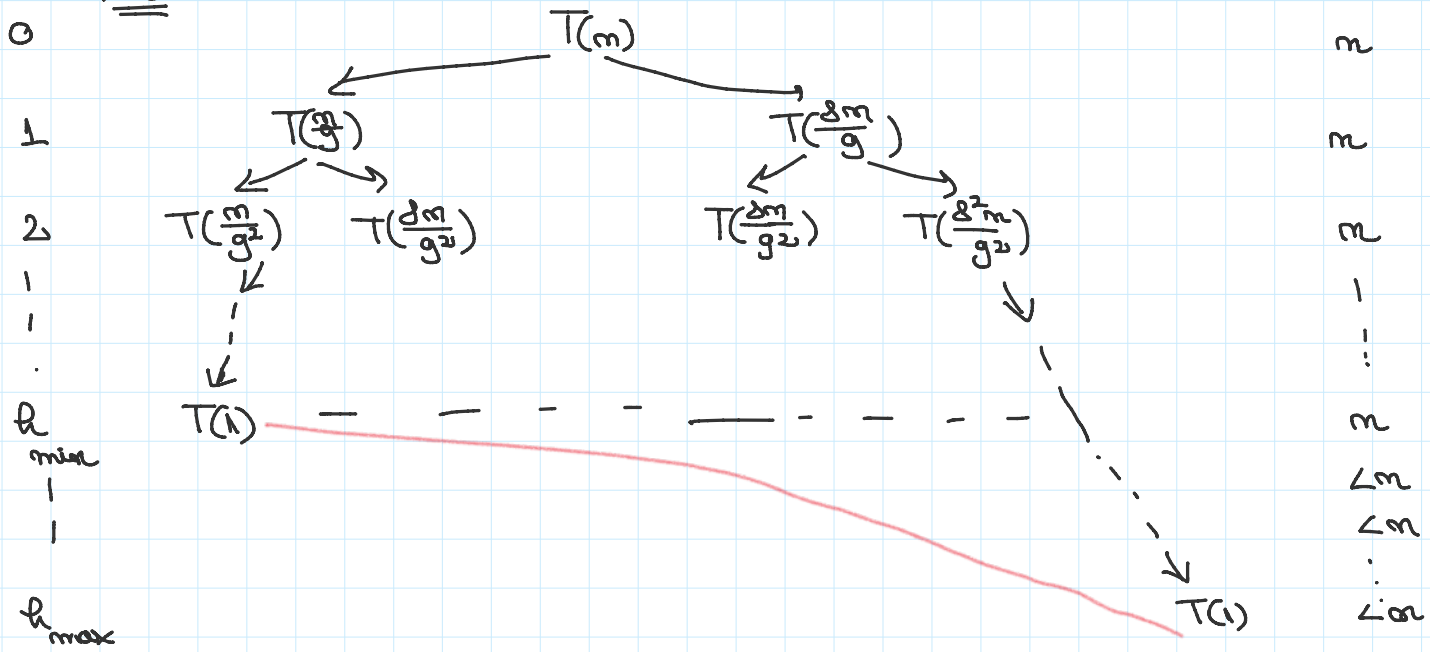
Wednesday, November 24, 2021 2:25 PM

Continuare Lab 6

(3)

→ $T(n) = T(n/9) + T(8n/9) + n$

Arb.



$\frac{n}{9^{h_{min}}} = 1 \Rightarrow h_{min} = \log_9 n$

$\frac{n}{(\frac{9}{8})^{h_{max}}} = 1 \Rightarrow h_{max} = \log_{9/8} n$

$S_{min} \leq T(n) \leq S_{max}$

$S_{min} = \sum_{i=0}^{h_{min}} cost_nivel = n \cdot h_{min} = n \log_9 n \Rightarrow S_{min} \in \Theta(n \log n)$

$S_{max} = \sum_{i=0}^{h_{max}} n = n \cdot h_{max} = n \log_{(9/8)} n \Rightarrow S_{max} \in \Theta(n \log n)$ ⇒

⇒ $T(n) \in \Theta(n \log n)$

→ $T(n) = T(2n/3) + 1$

Master

$\log_b a = \log_{3/2} 1 = 0$ | $\overline{\overline{=}}$ $T(n) \in \Theta(\log n)$

$f(n) = \Theta(n^0)$

Arbo.

0	$T(n)$	1
1	$T(\frac{2n}{3})$	1
2	$T(\frac{4n}{9})$	1
⋮	⋮	⋮
$\log_{3/2} n = k$	$T(1)$	1

$T(n) = \sum_{i=0}^{\log_{3/2} n} 1 = \log_{3/2} n + 1 \Rightarrow$

$\Rightarrow T(n) \in \Theta(\log n)$

Subst: $\exists c_1 \log_{3/2} n \leq T(n) \leq c_2 \log_{3/2} n$

(ii) $c_1 \log_{3/2} \frac{2n}{3} \leq T(\frac{2n}{3}) \leq c_2 \log_{3/2} \frac{2n}{3} \Leftrightarrow c_1 \log_{3/2} n - c_1 \leq T(\frac{2n}{3}) \leq c_2 \log n - c_2$

(Pi)

$c_1 \log n \leq c_1 \log n + 1 - c_1 \leq T(n) \leq c_2 \log n + 1 - c_2 \leq c_2 \log n$

Vrem $\begin{cases} 1 - c_1 \geq 0 \\ 1 - c_2 \leq 0 \end{cases} \Leftrightarrow \begin{cases} c_1 \leq 1 \\ c_2 \geq 1 \end{cases}$ OK

$\rightarrow$ $T(n) = T(\sqrt{n}) + 1$

Schimbare de var: $n = 2^k \Leftrightarrow k = \log_2 n$

$T(2^k) = T(2^{k/2}) + 1$

Notăm: $T(2^k) = F(k)$

$\Rightarrow$ $F(k) = F(\frac{k}{2}) + 1$

Master

$\log_b a = 0 = c \overset{\overline{\overline{=}}}{\Rightarrow} F(k) \in \Theta(\log k)$

$\Downarrow$

$T(2^k) \in \Theta(\log k)$

$T(n) \in \Theta(\log(\log n))$

$\rightarrow$ $T(n) = 2T(\sqrt{n}) + n$

Sch. var.: $n = 2^k \Leftrightarrow k = \log n$ și notăm $F(k) = T(2^k)$

$$T(2^k) = 2T(2^{k/2}) + 2^k$$

$$F(k) = 2F(\tfrac{k}{2}) + 2^k$$

Master: cazul III

+ verif că $\exists k < 1$ a.î. $af(\frac{k}{b}) < kf(k) \Leftrightarrow 2 \cdot 2^{k/2} < k \cdot 2^k$

$2^{1-\frac{k}{2}} < k$

$k = 0{,}9$ OK

$\Rightarrow F(k) \in \Theta(2^k)$

$T(2^k) \in \Theta(2^k)$

$T(n) \in \Theta(n)$

$\rightarrow T(n) = \sqrt{n}\, T(\sqrt{n}) + n$

Sch. var.: $2^k = n$

$$T(2^k) = 2^{k/2} T(2^{k/2}) + 2^k \quad | : 2^k$$

$$\frac{T(2^k)}{2^k} = \frac{T(2^{k/2})}{2^{k/2}} + 1$$

Notăm $F(k) = \frac{T(2^k)}{2^k}$

$F(k) = F(k/2) + 1$ $\overset{\text{Master III}}{\Rightarrow}$ $F(k) \in \Theta(\log k)$

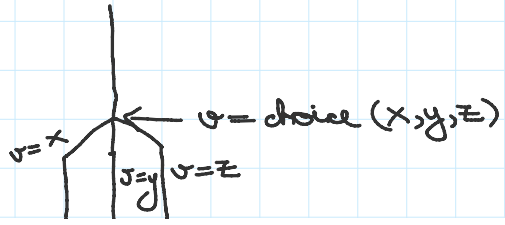
$\frac{T(2^k)}{2^k} \in \Theta(\log k)$

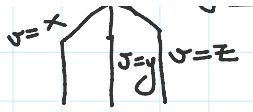
$T(2^k) \in \Theta(2^k \log k)$

$T(n) \in \Theta(n \log(\log n))$

Lab 7 - Nedeterminism

Exercitiu nedeterminist

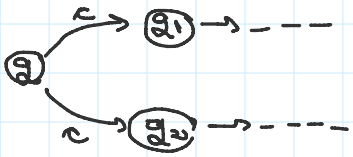




MTD



MTN

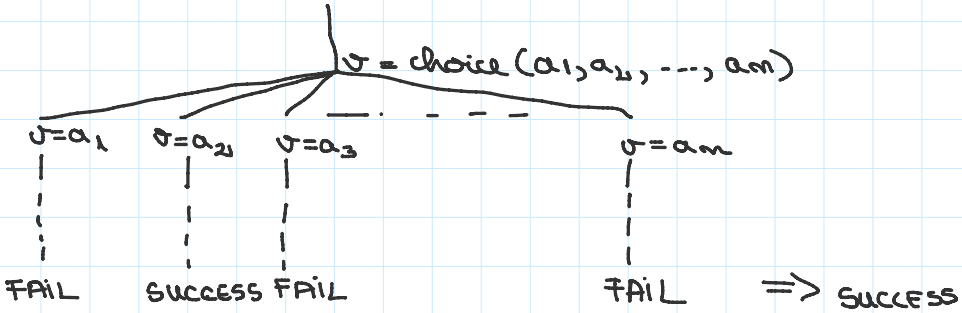


Notații pseudocod:

v = choice ($a_1, a_2, \dots, a_m$)
→ programul realizează n copii care rulează independent
→ în copia i, $v = a_i$

return SUCCESS
→ soluția a fost găsită

return FAIL
→ calea curentă nu repr. o soluție, dar algoritmul tot poate să întoarcă SUCCESS



Complexitate nedeterministă / angelică:

choice ($a_1, a_2, \dots, a_m$) // O(1)

= complexitatea celui mai scurt drum spre SUCCESS, dacă există SAU complexitatea celui mai lung drum spre FAIL, altfel

Ex.:

```
l = [] //O(1)
for i in range(0, n): //O(n)
    v = choice(0, 1) //O(1)
    l.append(v) //O(1)
```

=> Compl. angelică: O(n)

Exerciții

① SAT – "Satisfiability problem"

SAT(φ) = 1 (Adevărat/True) dacă $\exists$ o interpretare pt φ a.î. formula e adevărată

↑ formulă booleană în FNC

↓ o atribuire de 0/1 pt. fiecare variabilă din φ

FNC (Formă Normal Conjunctivă):

$\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m$ > C_i clauze

$C_i = L_{i1} \vee L_{i2} \vee \dots \vee L_{im}$ > L_{ij} sunt literali

$L_{ij} = X$ sau $L_{ij} = \bar{X}$ > X este o variabilă

→ SAT(φ) = ?

$\varphi = (\bar{X_0} \vee X_3 \vee X_5) \wedge X_3 \wedge (X_3 \vee X_6 \vee \bar{X_4} \vee X_0 \vee X_2 \vee X_1 \vee X_5) \wedge (X_5 \vee \bar{X_1} \vee \bar{X_3} \vee X_0) \wedge (\bar{X_1} \vee \bar{X_5} \vee X_6)$

X_0

X_1

X_2

X_3 1

X_4

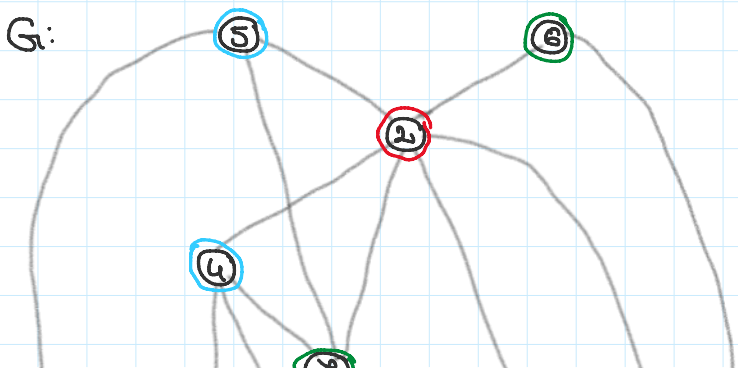
X_5 1

X_6 1

⇒ SAT(φ) = 1

② K-Color(G(V,E), k) = 1 dacă putem asigna fiecărui nod din V câte o "culoare" din $\{1, 2, 3, \dots, k\}$ a.î. 2 noduri adiacente au culori diferite

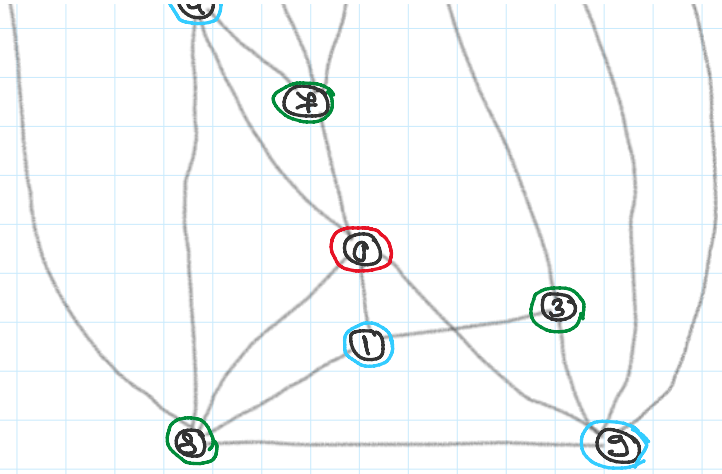
G:



K-Color(G, 4) = 1

Mai mult:

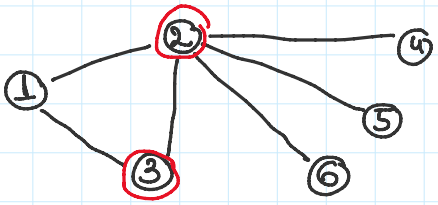
K-Color(G, 3) = 1



3

→ K-Vertex-Cover (G(V,E), k) = 1 dacă ∃ o acoperire formată din k noduri din V a.î. toate muchiile au cel puțin un capăt în acoperire

Ex:



k=2

{2,3} este un "vertex cover"

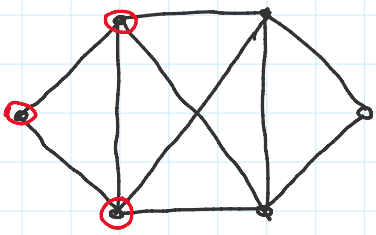
```
def kVertexCover(G(V, E), k):
    # formam toate acoperirile (submultimi din V de dim. k)
    cover = []
    for i in 1...k:
        # alegem un nod din V, pt. a-l adauga la acoperire
        nod = choice(V)
        V.remove(nod)
        cover.append(nod)

    # verificam daca acoperirea curenta este solutie sau nu
    for (u, v) in E:
        if u not in cover and v not in cover:
            return FAIL

    # toate muchiile au un capat in acoperire
    return SUCCESS
```

→ K-Clique (G(V,E), k) = 1 dacă ∃ un subgraf C format din k noduri a.î. ∀u,v∈C, (u,v)∈E

Ex. pt. k=3:



```
def kClique(G(V, E), k):
    # formam toate submultimile din V de dim. k
    clique = []
    for i in 1...k:
        # alegem un nod din V, pt. a-l adauga in clica
        nod = choice(V)
        V.remove(nod)
        clique.append(nod)

    # verificam daca clique este o solutie
    for u in clique:
        for v in clique:
            if u != v and (u, v) not in E:
                return FAIL

    # toate nodurile din clique sunt adiacente 2 cate 2
    return SUCCESS
```

→ K-Color (G(V,E), k)

Ex: la ex. 2

```
def kColor(G(V, E), k):
    # exploram toate asignarile posibile pt fiecare nod
    colors = {}
    for nod in V:
        c = choice(1..k)
        colocrs[nod] = c

    # verificam daca asignarea e o solutie
    for (u, v) in E:
        if colors[u] == colors[v]:
            return FAIL

    return SUCCESS
```

→ Partitioning (A) = 1 dacă ∃ $P_1, P_2 \subseteq A$, $P_1 \cup P_2 = A$, $P_1 \cap P_2 = \emptyset$ a.i. sum(P_1) = sum(P_2)

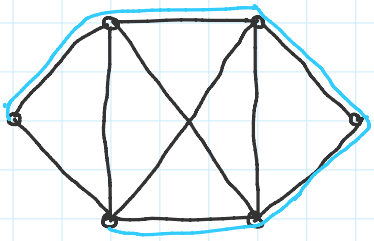
Ex: $A = \{1, 2, 3, 4\}$
$P_1 = \{1, 4\}$, sum(P_1) = 5
$P_2 = \{2, 3\}$, sum(P_2) = 5 → DA

```
def Partitioning(A):
    # cream toate partitionarile posibile
    P1 = []
    P2 = []
    for x in A:
        partition = choice(1, 2)
        if partition == 1:
            P1.append(x)
        if partition == 2:
            P2.append(x)

    # verific daca P1 si P2 sunt solutie
    if sum(P1) == sum(P2):
        return SUCCESS
    return FAIL
```

→ HamiltonianPath (G(V,E)) = 1 dacă ∃ un drum prin G care să treacă prin toate nodurile exact o dată

Ex:



```
def HamiltonianPath(G(V, E)):
    n = Len(V)

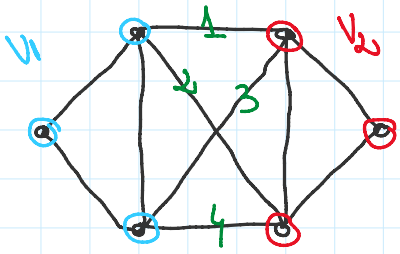
    # creez toate posibilitatile de drum de n noduri
    path = []
    for i in 1..n:
        nod = choice(V)
        path.append(nod)
        V.remove(nod)

    # verific daca path e o solutie
    for i in 1..n-1:
        if (path[i], path[i+1]) not in E:
            return FAIL

    return SUCCESS
```

→ K-Cut (G(V,E), k) = 1 dacă putem împărți V în 2 submulțimi a.i. ∃ k muchii care au capetele în submulțimi diferite

Ex. pt. k = 4 :




```python
def kCut(G(V, E), k):
    V1 = []
    V2 = []
    for nod in V:
        x = choice(1, 2)
        if x == 1:
            V1.append(nod)
        if x == 2:
            V2.append(nod)

    # verific daca partitionarea V1, V2 este solutie
    # (pt asta, numar nr. de muchii de unesc partitiile)
    nr = 0
    for (u, v) in E:
        if (u in V1 and v in V2) or (u in V2 and v in V1):
            nr += 1

    if nr >= k:
        return SUCCESS
    return FAIL
```